

Posterior Sampling for Anytime Motion Planning on Graphs with Expensive-to-Evaluate Edges

Brian Hou, Sanjiban Choudhury, Gilwoo Lee, Aditya Mandalika, and Siddhartha S. Srinivasa

Abstract—Collision checking is a computational bottleneck in motion planning, requiring lazy algorithms that explicitly reason about when to perform this computation. Optimism in the face of collision uncertainty minimizes the number of checks before finding the shortest path. However, this may take a prohibitively long time to compute, with no other feasible paths discovered during this period. For many real-time applications, we instead demand strong anytime performance, defined as minimizing the cumulative lengths of the feasible paths yielded over time. We introduce Posterior Sampling for Motion Planning (PSMP), an anytime lazy motion planning algorithm that leverages learned posteriors on edge collisions to quickly discover an initial feasible path and progressively yield shorter paths. PSMP obtains an expected regret bound of $\tilde{O}(\sqrt{SAT})$ and outperforms comparative baselines on a set of 2D and 7D planning problems.

I. INTRODUCTION

We formalize the problem of anytime motion planning. Existing algorithms typically make *asymptotic* guarantees [1] that they will eventually find the optimal path. However, such analysis leaves several practical questions unanswered. Given a budget of computation time, how sub-optimal will the resulting path be? How will increasing the computation budget improve the quality of the solution? Formalizing these questions helps us better understand important *anytime* properties, not just asymptotic properties. This will also enable practitioners to make more informed choices about the algorithms they deploy.

We focus on anytime planning on fixed graphs.¹ Here, vertices are sampled robot configurations and edges are potential robot motions. Evaluating if an edge is in collision is *computationally expensive* [2]. Hence, search algorithms must be lazy [3], i.e., minimize edge evaluation as they search for paths. Our goal is to quickly find feasible paths and shorten them as time permits—we refer to this as lazy anytime search [4].

What if such an algorithm was provided a posterior distribution of edge collisions? This could either be based on a dataset of prior experience or domain knowledge about obstacle geometries. The search must consider two factors: the length of a path and the likelihood of it being in collision. A desirable outcome, shown in Fig. 1, is to initially evaluate

longer paths that have lower probability of collision. Eventually, as uncertainty collapses, the search evaluates shorter and shorter paths. This strategy encapsulates a fundamental trade-off: it can either explore shorter paths to potentially improve future performance or exploit the most likely path to attain better immediate performance.

We formalize this within the framework of Bayesian Reinforcement Learning (BRL). We first define lazy search on a graph as solving a deterministic, goal-directed, Markov Decision Process (MDP) where rewards (collision status) are unknown. A BRL algorithm explores this MDP as it attempts to find the optimal policy (path). To judge how quickly an algorithm learns, we consider the bandit setting [5]: in each round of learning, an agent pulls an arm (evaluates a path), receives a loss (negative of path length), and accumulates regret with respect to the optimal arm. A low expected regret [6] corresponds to evaluating edges that not only lead to shorter paths, but also drive down uncertainty over time. Hence, our key insight is:

Good anytime search performance is equivalent to minimizing *Bayesian regret*.

However, the space of paths is combinatorially large, which makes many bandit algorithms that require explicit posteriors inapplicable. Fortunately, while explicitly computing this posterior is hard, sampling from it is quite easy! Posterior sampling offers strong guarantees on Bayesian regret [7]. Our algorithm, Posterior Sampling for Motion Planning (PSMP), samples a graph from the posterior and only evaluates edges along the shortest path in that graph. It is both simple to implement and—given a posterior to sample from—free of tuning parameters. We make the following contributions.

- We introduce a novel formulation of anytime search on graphs as an instance of Bayesian Reinforcement Learning (Section III).
- We introduce a general framework, Experienced Lazy Path Search, that unifies several existing search algorithms that leverage prior experience (Section IV).
- We show that PSMP has good theoretical anytime performance by bounding its Bayesian regret (Section IV).
- We demonstrate that PSMP effectively leverages posteriors to outperform comparative baselines on a set of 2D and 7D motion planning problems (Section V).

This work was (partially) funded by the National Institute of Health R01 (#R01EB019335), National Science Foundation CPS (#1544797), National Science Foundation NRI (#1637748), the Office of Naval Research, the RCTA, Amazon, and Honda Research Institute USA. Brian Hou is partially supported by a NASA Space Technology Research Fellowship. Gilwoo Lee is partially supported by Kwanjeong Educational Foundation.

All authors are with the Paul G. Allen School of Computer Science & Engineering, University of Washington, Seattle, WA 98195 {bhous, sanjibac, gilwoo, adityavk, siddh}@cs.uw.edu

¹ While analyzing anytime algorithms that continue to sample the configuration space is the eventual goal, obtaining meaningful bounds requires analyzing the nature of probability distributions over continuous configuration space geometry. This becomes quite challenging, even for simple geometries, and hence is currently out of scope.

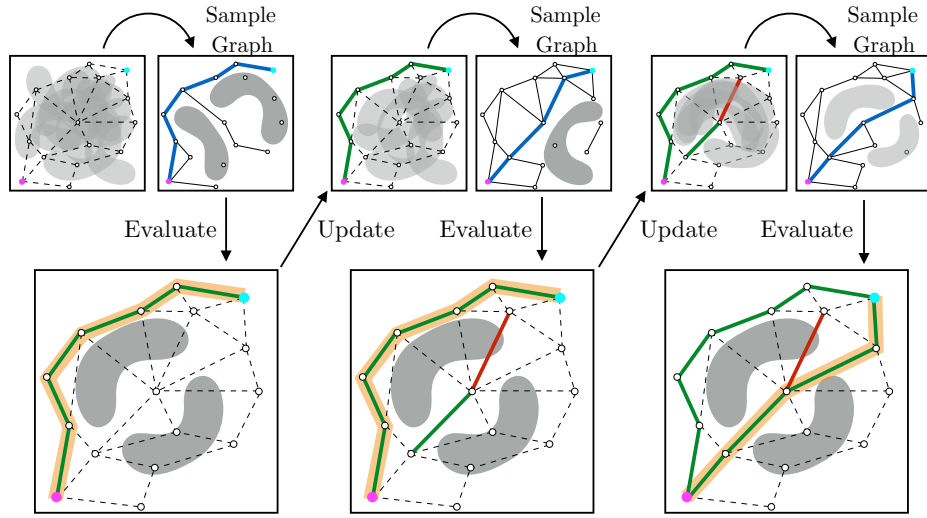


Fig. 1: Posterior Sampling for Motion Planning is a Bayesian anytime motion planning algorithm. The graph's edge collision statuses are initially unknown (dashed). In each iteration of PSMP, the posterior (top left) is sampled to produce a graph (top right). The sampled graph's shortest path (blue) is evaluated for collisions against the real world (bottom). Edges are either found to be in collision (red) or collision-free (green); those statuses are used to update the posterior. If all edges in the proposed path are collision-free, PSMP updates its current shortest path (yellow), which can be emitted at any time.

II. RELATED WORK

A. Priors in Lazy Search

Planning with expensive collision-checking is a well-studied problem in motion planning. Lazy search approaches deal with this by only checking edges that lie along the estimated shortest path [3, 8] or the shortest subpath [9, 10]. For real-world robotics problems, leveraging priors on edge collisions can produce significant speed-ups. FuzzyPRM [11] evaluates paths that are most likely to be feasible. GLS [12] uses priors to quickly invalidate subpaths until the shortest path is found. STROLL [13] learns an edge evaluation policy for LazySP. BiSECT [14] and DiRECT [15] formalize Bayesian motion planning and compute near Bayes-optimal policies for finding feasible paths. However, these approaches do not aim for anytime performance.

Several methods model collision posteriors to exploiting structure in planning. One approach is to predict validity of unevaluated edges given the outcomes of evaluated edges [16]. Other approaches try to model the configuration space belief given observed collisions and guide search with that belief [17–20]. However, these approaches do not directly aim to approximate the Bayesian posterior.

B. Anytime Planning

For many real-time planning applications, an algorithm must be able to deal with an unknown planning time budget. This is achieved by incremental sampling methods, such as RRT* [1] or RRT++ [21], which guarantee asymptotic optimality. However, they make no promises on convergence rate and are often slow in practice. Incremental densification techniques, on the other hand, offer provable speed-ups by restricting new samples to a region that can only improve the current solution [22–24]. However, these methods cannot provably exploit priors on the configuration space.

Another way of viewing anytime planning is through the lens of heuristic search on large graphs. Weighted A*

search with an inflated heuristic finds feasible paths quickly, although the solution may be suboptimal. Anytime variants of A* [4, 25] efficiently run a succession of weighted A* searches with decreasing inflation. However, heuristics may not always indicate existence of feasible paths. POMP [26] uses priors on edge validity to explicitly trade-off path likelihood and path length. AEE* [27] uses Bernoulli priors on edges to generate a set of plausible shortest paths, which is then evaluated in an anytime fashion. However, these do not offer guarantees for arbitrary priors.

C. Bayesian Reinforcement Learning (BRL)

Standard RL approaches consider optimal exploration of an unknown MDP until an optimal policy is computed. In the absence of prior knowledge, PAC-MDP [28] approaches result in exhaustive experimentation in every possible state. BRL [29] introduces a prior on rewards and transitions, requiring only enough exploration to find a good policy in expectation. Since Bayes-optimality is intractable, this can only be solved approximately [30, 31]. An alternative to Bayes-optimality is Bayesian regret, which views the learning as an online process of interacting with MDP. This leads to simpler algorithms such as UCRL2 [32] and Posterior Sampling RL [7]. We build on [7] to bound Bayesian regret for the problem of anytime planning.

III. BAYESIAN ANYTIME MOTION PLANNING

We assume a fixed explicit graph $G = \{V, E\}$, where V denotes a set of vertices and E a set of edges.

Given start and goal $(v_s, v_g) \in V$, a path ξ is a sequence of connected vertices $(v_1, v_2, \dots, v_l)$, $v_1 = v_s, v_l = v_g$. Let $w : E \rightarrow \mathbb{R}^+$ be the weight of an edge. The length of a path is the sum of edge weights, i.e. $w(\xi) = \sum_{e \in \xi} w(e)$. We define a world $\phi \in \mathbb{R}^{|E|}$ as the vector of edge weights. The weights are unknown and discovered by edge evaluation, which is *computationally expensive*. Hence, an algorithm's planning time is determined by the edges that it evaluates.

As a planning algorithm evaluates edges $e_1, \dots, e_N$, it uncovers a series of progressively shorter paths $\xi_1, \dots, \xi_N$. The objective of anytime planning is to minimize the cumulative length of paths, i.e., $\sum_{i=1}^N w(\xi_i)$.

We consider a Bayesian setting where we have a prior distribution on worlds $P(\phi)$ obtained from past experience. As an algorithm evaluates edges, let $\psi_t = \{w(e_1), \dots, w(e_t)\}$ be the history of observations, i.e., outcome of edge evaluations. Given this history, a Bayesian planning algorithm can compute a posterior $P(\phi|\psi_t)$ to decide which path to evaluate. The objective of a *Bayesian anytime planning* algorithm is to minimize the expected cumulative length of paths computed given the prior over the worlds $P(\phi)$.

We will now establish an equivalence between our problem and the repeated episodic BRL problem described in [7]. We consider a deterministic finite horizon MDP $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, R^{\mathcal{M}}, T, \tau, s_1 \rangle$. A state $s \in \mathcal{S}$ corresponds to a vertex $v \in V$, actions $\mathcal{A}(s)$ correspond to the set of adjacent edges, and transition function $s' = T(s, a)$ is the adjacency matrix of the graph. The reward function $R^{\mathcal{M}}(s, a)$ is 0 if $s = v_g$, else it is $-w(e)$, where e is the edge associated with a . The horizon τ corresponds to the maximum number of edges in a path. The initial state s_1 is v_s .

The solution to the MDP is a partial policy $\mu : \mathcal{S} \rightarrow \mathcal{A}$ corresponding to a path ξ . The policy's value is $V_{\mu}^{\mathcal{M}}(s_1) = \sum_{j=1}^{\tau} R^{\mathcal{M}}(s_j, \mu(s_j))$ is the negative path length $-w(\xi)$.

For an unknown MDP $\mathcal{M}^*$, the reward function $R^{\mathcal{M}^*}$ is unknown. The prior over worlds $P(\phi)$ maps to a prior $P(R^{\mathcal{M}})$. A learning algorithm must infer the reward function by repeatedly interacting with $\mathcal{M}^*$. In each episode $i = 1, \dots, m$, it executes policies $\mu_1, \dots, \mu_m$, updates history ψ_i , and tracks the best discovered policy $\hat{\mu}_i = \max_{j=1, \dots, i} V_{\mu_j}^{\mathcal{M}^*}(s_1)$. We define an algorithm's regret to be the cumulative difference between the value of the optimal policy and the best discovered policy after each episode

$$\text{REGRET}(m) = \sum_{k=1}^m \Delta_k, \text{ where } \Delta_k = V_{\mu^*}^{\mathcal{M}^*}(s_1) - V_{\hat{\mu}_k}^{\mathcal{M}^*}(s_1).$$

The objective of the BRL problem in [7] is to minimize the expected regret $\mathbb{E}_{P(R^{\mathcal{M}})}[\text{REGRET}(m)]$, also known as the Bayesian regret $\mathbb{E}[\text{REGRET}(m)]$. Note that this is a constant offset from the objective we defined in Bayesian anytime planning, i.e., $\mathbb{E}_{P(R^{\mathcal{M}})}[\text{REGRET}(m)] = \mathbb{E}_{P(\phi)}[\sum_{k=1}^m w(\xi_k) - w(\xi^*)]$.

Although we formulate the problem more generally, we focus on a specific instantiation where each edge has a binary collision status. Edge evaluation corresponds to collision checking the edge. If an edge is not in collision, the weight $w(e)$ is the distance between the two vertices, which is known. If the edge is in collision, $w(e)$ is set to a large value $C_{\max}$. For compactness, we redefine a world to be the collision status of all edges $\phi \in \{0, 1\}^{|E|}$, effectively binarizing the problem. A feasible path has $\phi(e) = 1, \forall e \in \xi$.

The MDP we have defined allows us to establish equivalences between RL and other lazy motion planning formulations proposed in previous work. The *lazy shortest path problem* [3], where the shortest feasible path must be found

Algorithm 1 Experienced Lazy Path Search

Require: Graph G , Prior $P(\phi)$, Proposer $\text{ComputePath}(\cdot)$

- 1: Initialize history $\psi \leftarrow \emptyset$, evaluated edges $E_{\text{eval}} \leftarrow \emptyset$
 - 2: **while** termination criteria not met **do**
 - 3: Invoke proposer $\xi = \text{ComputePath}(G, P(\phi|\psi))$.
 - 4: **while** path ξ is not invalid **and** ξ is unevaluated **do**
 - 5: Evaluate unevaluated edge with highest
 - 6: posterior collision probability
 - 7: $e^* = \arg \min_{e \in \xi \setminus E_{\text{eval}}} P(\phi(e) = 1|\psi)$.
 - 8: Add edge to evaluated set $E_{\text{eval}} \leftarrow E_{\text{eval}} \cup \{e^*\}$.
 - 9: Update history ψ with outcomes.
 - 10: **if** ξ is valid **then** emit ξ .
-

while eliminating all shorter paths, is equivalent to the PAC-MDP [28] problem of optimally exploring an MDP until an optimal policy is found. Similarly, the Bayesian version of this problem is equivalent to PAC-BAMDP [29]. The *feasible path problem* [14] is equivalent to Bayes-optimally exploring the MDP until a valid policy is found.

IV. EXPERIENCED LAZY PATH SEARCH

We present a general framework for experienced lazy search that uses priors on edge validities to minimize collision checking. This unifies search for anytime planning, as well as other objectives such as efficiently finding the shortest path or any feasible path. We then introduce *Posterior Sampling for Motion Planning* (PSMP), a new algorithm that bounds expected anytime planning performance.

A. Experienced Lazy Path Search

We begin by presenting a framework for lazy search algorithms that uses priors, thus unifying several previous works in this area [3, 13, 14, 26]. In Experienced Lazy Path Search (Algorithm 1), a *proposer* lazily computes a path from the start to goal (without any edge evaluation) and a *path validator* chooses edges along the path to evaluate.²

We fix the path validator to the *FailFast* rule [12] for all proposers. This rule tries to invalidate a proposed path as quickly as possible, formally stated as follows:

Theorem 1: The *FailFast* validator repeatedly evaluates the edge with highest probability of collision, until one edge is found to be in collision or all edges are found to be collision-free. This is optimal for eliminating a single candidate path³, if prior $P(\phi)$ is independent Bernoulli. For general priors, this is near-optimal with a factor of 4.

Proof: This can be mapped to a Bayesian search problem where the goal is to sequentially search for an item (invalid edge) in a set of boxes (unevaluated edges) while minimizing cost of search. Bounds follow from [33]. ■

²Note this unifying framework differs from the framework in Generalized Lazy Search (GLS) [12]. First, GLS looks at problems where planning time depends on *both* graph operations and edge evaluations. Hence, it argues for *interleaving* search with evaluation of *sub-paths*. Second, GLS exclusively considers the shortest path problem.

³However, it ignores overlap among paths unlike [15] for simplicity.

TABLE I: Different algorithms as instantiations of Experienced Lazy Path Search.

Algorithm	ComputePath($G, P(\phi \psi)$)	Performance Guarantee
PSMP (ours)	Sample a world $\phi \sim P(\phi \psi)$, return ξ^*	Anytime (Bayesian regret)
LAZYSP [3]	Generate optimistic world ϕ , return ξ^*	Shortest path (OFU)
MAXPROB [14]	Set weights $-\log P(\phi(e) = 1 \psi)$, return ξ^*	Feasible path (Bayes-optimal)
POMP [26]	Set weights $\alpha w(e) - (1 - \alpha) \log P(\phi(e) = 1 \psi)$, return ξ^*	Anytime (Pareto optimality)

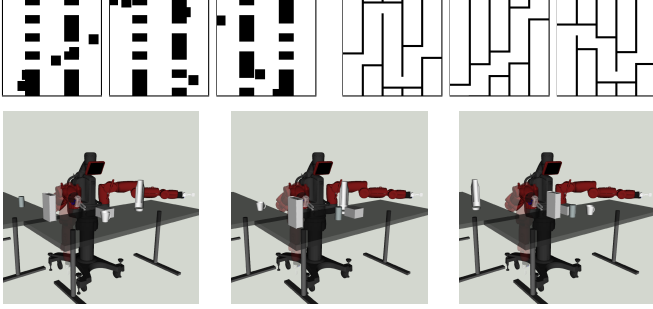


Fig. 2: Example problems from two 2-DOF datasets (top) and the 7-DOF dataset (bottom). In the 2-DOF datasets, a point robot navigates from the bottom left corner to the top right. In the 7-DOF dataset, the robot arm moves from below the table (transparent) to above, while avoiding clutter.

By varying the proposer $\text{ComputePath}(\cdot)$, we can recover several algorithms from the literature that aim for different performance guarantees. All proposers listed in Table I view G with modified edge weights and propose the shortest path on the modified graph. LAZYSP [3] returns the optimistic shortest path by modifying all unevaluated edges in G to be feasible. It eliminates candidate paths in order of increasing length and terminates after a feasible path is found, yielding an OFU-like guarantee of finding the shortest path with minimal evaluations. MAXPROB [14] returns the most likely feasible path by modifying the weights to be negative log likelihood of validity. It sequentially evaluates the most probable path, terminating after a feasible path is found. Similar to Theorem 1, MAXPROB Bayes-optimally proposes the fewest paths before finding a feasible path if prior $P(\phi)$ is independent Bernoulli and near Bayes-optimally (factor of 4) otherwise. Finally, POMP [26] balances edge weight with the likelihood of being collision free. Increasing α between iterations of Algorithm 1 traces out the Pareto frontier of the two objectives, starting with the most probable path while guaranteeing asymptotic optimality. However, this anytime property comes without guarantees on rate of improvement.

B. Posterior Sampling for Motion Planning (PSMP)

PSMP aims to guarantee anytime behavior. It essentially borrows the idea of posterior sampling [7], or Thompson sampling [34], and applies it in the space of paths. PSMP samples a world ϕ from the posterior distribution $P(\phi|\psi)$ conditioned on the history ψ . It then computes the shortest path ξ^* on ϕ and proposes it for evaluation.

We will now establish Bayesian regret bounds for PSMP, following the analysis of posterior sampling for reinforcement learning [7] and multi-armed bandits [35]. From an algorithmic perspective, PSMP is attractive because it requires solving only a single shortest path problem. By contrast, other Bayesian search methods like Monte Carlo

Tree Search [36] or even heuristics like QMDP [37] require several calls to the search. PSMP also requires no tuning parameters.

By sampling paths according to the posterior probability they are optimal, PSMP continues to sample plausible shortest paths. As PSMP gains more information, the posterior concentrates around the true world. The regret for PSMP grows sublinearly as $\tilde{O}(\sqrt{SAT})$ where T is the total number of timesteps, matching the lower bound from [32]. For the analysis, we assume edge weights are normalized $[0, 1]$.

Theorem 2: The expected regret is bounded as

$$\mathbb{E}[\text{REGRET}(T)] = O(\tau \sqrt{SAT \log(SAT)}) \quad (1)$$

Proof: We follow the analysis of [7], adapted for the special case of a deterministic MDP to obtain tighter regret bounds. We refer the reader to Appendix A for details. ■

C. Estimating Edge Collision Posteriors

In our experiments, we consider two possible approaches for estimating the posterior distribution $P(\phi|\psi)$. If there is no dataset of previous planning problems to learn from, the collision-checked configurations from ψ can inform a nearest neighbor-based posterior for the current problem [18]. We find that only considering the 1-nearest neighbor q_{near} produces the best collision estimates due to massive label imbalance in favor of collision-free points.

We differ from [18] by assuming a uniform Beta(1, 1) prior on configuration space collision probability. The status of the nearest neighbor counts as a partial success or failure with weight $\exp(-\eta \|q - q_{near}\|)$. The expected posterior probability that configuration q is free is then

$$\mathbb{E}[P(\phi(q) = 0|\psi)] = \frac{\exp(-\eta \|q - q_{near}\|) \mathbb{1}[\phi(q_{near})=0] + 1}{\exp(-\eta \|q - q_{near}\|) + 2}.$$

To estimate the posterior probability that an edge is collision-free, we take the minimum collision-free probability of discretized points along the edge.

Alternatively, if we know that worlds are uniformly drawn from a finite set of possible worlds, we can precompute the collision statuses for every edge in the graph against every world ϕ . Then, the posterior is simply uniform over the remaining set of worlds that are consistent with ψ .

This finite set posterior is one example of how planning algorithms may be able to leverage the structure existing in everyday environments. The configuration space nearest-neighbor posterior only assumes that nearby configurations will have similar labels, which is a more broadly applicable (but less informative) structure. This makes it well-suited for novel environments where the posterior does not have problem examples to learn from.

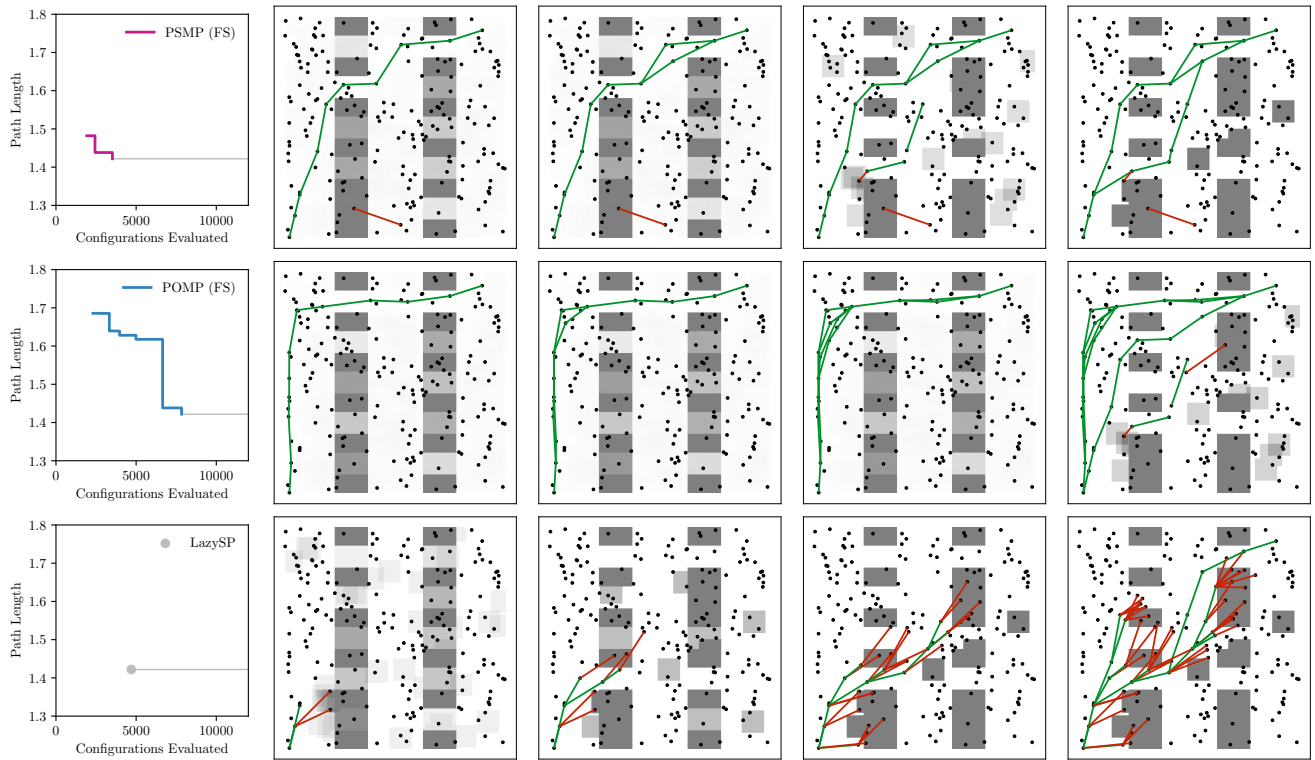


Fig. 3: (Left) Anytime performance of PSMP (top), POMP (middle), and LAZYSP (bottom). The gray line corresponds to the environment’s shortest path. (Right) Snapshots of each algorithm’s progress, with the finite set posterior used by PSMP and POMP. Regions with higher probability of collision are colored with darker shades of gray. Evaluated edges are either found to be in collision (red) or collision-free (green).

V. EXPERIMENTS

We evaluate the anytime performance of PSMP on 2-DOF [14] and 7-DOF motion planning datasets (Fig. 2). The 7-DOF manipulator dataset was generated by randomly perturbing objects from an initial cluttered environment [38]. For 2-DOF problems, we compare PSMP and POMP, with both the nearest neighbor-based (NN) and finite set (FS) posterior variants from Section IV-C. We have highlighted their performance on two datasets here, and refer the reader to Appendix C for results on five more. For 7-DOF problems, we only consider the finite set posterior for both PSMP and POMP. In this domain, we additionally combine RRT-Connect with path shortening, a commonly-used heuristic for refining an initial feasible path [39]. Because collision checking dominates planning time, we report the number of configurations checked by each algorithm. We refer the reader to Appendix B for further experimental details.

We visualize sample runs by PSMP, POMP, and LAZYSP in Fig. 3. Note that since LAZYSP is not an anytime algorithm, it only produces one solution. Using the same finite set posterior as POMP, PSMP finds a shorter feasible path with fewer collision checks. Furthermore, it returns the shortest path faster than the uninformed LAZYSP baseline. POMP carefully attempts to avoid edges that may be in collision; as a result, refining the initial feasible solution can take a substantial amount of time.

The performance of these algorithms on the remainder of the test set (200 environments) is summarized in the first row of Fig. 4. The anytime performance of the nearest

neighbor-based (NN) and finite set (FS) posteriors have been separated for clarity. PSMP with the FS posterior has a marked improvement over the NN variant: it finds initial feasible solutions faster than all other algorithms and quickly refines them. However, averaging the worlds in the feasible set that are consistent with the evaluation history—as POMP does—can lead to over-exploration of impossible scenarios and degraded performance.

In the maze environments (Fig. 4, middle), PSMP with the FS posterior continues to outperform other algorithms. However, the evaluation history quickly narrows the posterior to a single feasible world, so POMP enjoys similar performance to PSMP. In these settings where there are very few feasible paths, PSMP with the NN posterior does not perform as well. We believe that this is because an anytime objective implicitly assumes the existence of multiple feasible paths. Although posterior sampling explores options quickly, exploration may be unnecessary if this assumption is violated. In such scenarios, a posterior that captures more global correlations in the environment may be needed for improved performance relative to an algorithm like LAZYSP.

On the 7-DOF manipulator environments, we chose to compare with heuristically shortening an initial feasible RRT path (RRT+PS) rather than RRT* [1]; although the latter guarantees asymptotic optimality, it empirically takes much longer to find initial solutions (Appendix C). While RRT+PS finds a feasible path faster than LAZYSP, it needs more collision checks and emits longer paths than PSMP or POMP with the FS posterior (Fig. 4, bottom).

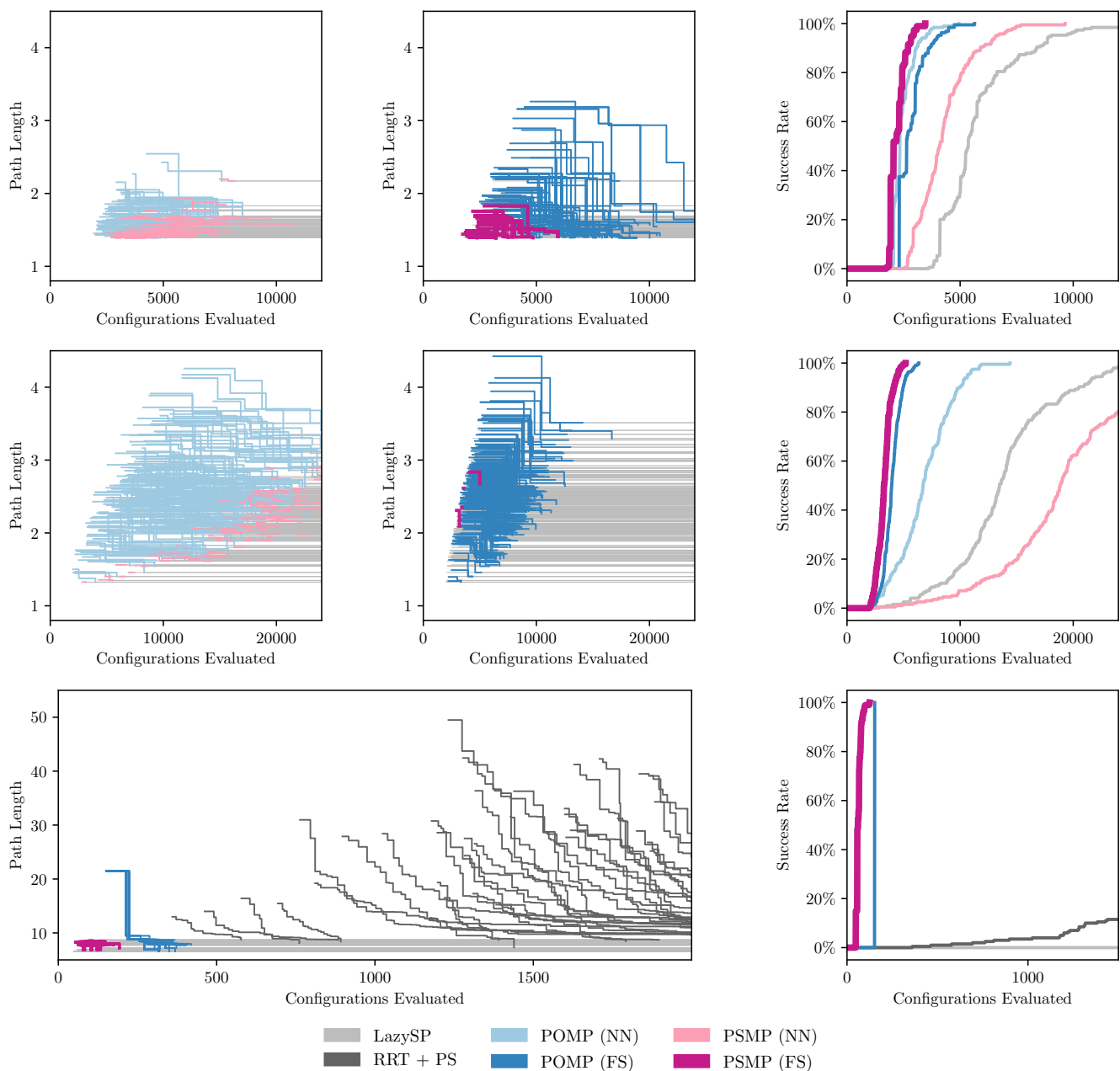


Fig. 4: (Left) Length of the best feasible path discovered by each anytime algorithm over time. (Right) Collision checking budget versus the percentage of planning problems where that budget is sufficient to discover a feasible path. (Top and Middle) 2-DOF environments corresponding to (Fig. 2, Left and Right). (Bottom) 7-DOF robot manipulator environment, corresponding to (Fig. 2, Bottom).

VI. DISCUSSION AND FUTURE WORK

Anytime algorithms should rapidly discover progressively shorter paths. We have formalized this intuitive objective as minimizing Bayesian regret. Sublinear Bayesian regret—which PSMP achieves—implies asymptotic optimality, while demanding good intermediate performance. We hope that drawing this connection between anytime motion planning and Bayesian reinforcement learning will open the door to further regret analysis of anytime algorithms.

In this work, we have focused on the problem of anytime search on a fixed graph. Many existing anytime algorithms take an incremental densification approach, requiring new

samples from the configuration space for continued improvement. Regret analysis for these continuous-space problems is an open challenge.

Empirically, PSMP has strong performance and improves further when more knowledge is incorporated into the posterior structure. Using data from previous planning environments to learn the underlying structure of collision posteriors, via e.g. unsupervised generative models, will enable PSMP to quickly solve new instances of those problems.

VII. APPENDICES

This paper is available with appendices at <https://arxiv.org/abs/2002.11853>.

REFERENCES

- [1] S. Karaman and E. Frazzoli. Sampling-based algorithms for optimal motion planning. *The International Journal of Robotics Research*, 30(7):846–894, 2011.
- [2] K. Hauser. Lazy collision checking in asymptotically-optimal motion planning. In *IEEE International Conference on Robotics and Automation*, 2015.
- [3] C. Dellin and S.S. Srinivasa. A unifying formalism for shortest path problems with expensive edge evaluations via lazy best-first search over paths with edge selectors. In *International Conference on Automated Planning and Scheduling*, 2016.
- [4] M. Likhachev, G.J. Gordon, and S. Thrun. ARA*: Anytime A* with provable bounds on sub-optimality. In *Advances in Neural Information Processing Systems*, 2004.
- [5] S. Bubeck and N. Cesa-Bianchi. Regret analysis of stochastic and nonstochastic multi-armed bandit problems. *Foundations and Trends® in Machine Learning*, 5(1):1–122, 2012.
- [6] J. Hannan. Approximation to Bayes risk in repeated play. *Contributions to the Theory of Games*, 3:97–139, 1957.
- [7] I. Osband, D. Russo, and B. Van Roy. (More) efficient reinforcement learning via posterior sampling. In *Advances in Neural Information Processing Systems*, 2013.
- [8] R. Bohlin and L.E. Kavraki. Path planning using lazy PRM. In *IEEE International Conference on Robotics and Automation*, 2000.
- [9] B. Cohen, M. Phillips, and M. Likhachev. Planning single-arm manipulations with n-arm robots. In *Eighth Annual Symposium on Combinatorial Search*, 2015.
- [10] A. Mandalika, O. Salzman, and S.S. Srinivasa. Lazy Receding Horizon A* for Efficient Path Planning in Graphs with Expensive-to-Evaluate Edges. In *International Conference on Automated Planning and Scheduling*, 2018.
- [11] C.L. Nielsen and L.E. Kavraki. A 2 level fuzzy PRM for manipulation planning. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2000.
- [12] A. Mandalika, S. Choudhury, O. Salzman, and S.S. Srinivasa. Generalized Lazy Search for Robot Motion Planning: Interleaving Search and Edge Evaluation via Event-based Toggles. In *International Conference on Automated Planning and Scheduling*, 2019.
- [13] M. Bhardwaj, S. Choudhury, B. Boots, and S. Srinivasa. Leveraging Experience in Lazy Search. In *Robotics: Science and Systems*, 2019.
- [14] S. Choudhury, S. Javdani, S. Srinivasa, and S. Scherer. Near-optimal edge evaluation in explicit generalized binomial graphs. In *Advances in Neural Information Processing Systems*, 2017.
- [15] S. Choudhury, S.S. Srinivasa, and S. Scherer. Bayesian active edge evaluation on expensive graphs. In *International Joint Conference on Artificial Intelligence*, 2018.
- [16] J.M. Esposito and J.N. Wright. Matrix completion as a post-processing technique for probabilistic roadmaps. In *Workshop on the Algorithmic Foundations of Robotics*, 2016.
- [17] B. Burns and O. Brock. Sampling-based motion planning using predictive models. In *IEEE International Conference on Robotics and Automation*, 2005.
- [18] J. Pan, S. Chitta, and D. Manocha. Faster sample-based motion planning using instance-based learning. In *Workshop on the Algorithmic Foundations of Robotics*, 2012.
- [19] J. Huh and D.D. Lee. Learning high-dimensional mixture models for fast collision detection in rapidly-exploring random trees. In *IEEE International Conference on Robotics and Automation*, 2016.
- [20] B. Lacevic, D. Osmankovic, and A. Ademovic. Burs of free C-space: a novel structure for path planning. In *IEEE International Conference on Robotics and Automation*, 2016.
- [21] Y. Abbasi-Yadkori, J. Modayil, and C. Szepesvari. Extending rapidly-exploring random trees for asymptotically optimal anytime motion planning. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2010.
- [22] D. Ferguson and A. Stentz. Anytime RRTs. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2006.
- [23] J.D. Gammell, S.S. Srinivasa, and T.D. Barfoot. Informed RRT*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2014.
- [24] J.D. Gammell, S.S. Srinivasa, and T.D. Barfoot. Batch Informed Trees: Sampling-based optimal planning via heuristically guided search of random geometric graphs. In *IEEE International Conference on Robotics and Automation*, 2015.
- [25] J. van den Berg, R. Shah, A. Huang, and K. Goldberg. ANA*: Anytime nonparametric A*. In *AAAI Conference on Artificial Intelligence*, 2011.
- [26] S. Choudhury, C. Dellin, and S.S. Srinivasa. Pareto-optimal search over configuration space beliefs for anytime motion planning. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2016.
- [27] V. Narayanan and M. Likhachev. Heuristic Search on Graphs with Existence Priors for Expensive-to-Evaluate Edges. In *International Conference on Automated Planning and Scheduling*, 2017.
- [28] A.L. Strehl, L. Li, and M.L. Littman. Reinforcement learning in finite MDPs: PAC analysis. *Journal of Machine Learning Research*, 10(Nov):2413–2444, 2009.
- [29] M. Ghavamzadeh, S. Mannor, J. Pineau, and A. Tamar. Bayesian Reinforcement Learning: A Survey. *Foundations and Trends® in Machine Learning*, 8(5-6):359–483, 2015.
- [30] J.Z. Kolter and A.Y. Ng. Near-Bayesian exploration in polynomial time. In *International Conference on Machine Learning*, 2009.
- [31] M. Chen, E. Frazzoli, D. Hsu, and W.S. Lee. POMDP-lite for Robust Robot Planning under Uncertainty. In *IEEE International Conference on Robotics and Automation*, 2016.
- [32] T. Jaksch, R. Ortner, and P. Auer. Near-optimal regret bounds for reinforcement learning. *Journal of Machine Learning Research*, 11(Apr):1563–1600, 2010.
- [33] A. Dor, E. Greenshtein, and E. Korach. Optimal and myopic search in a binary random vector. *Journal of applied probability*, 1998.
- [34] W.R. Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3/4):285–294, 1933.
- [35] D. Russo and B. Van Roy. Learning to optimize via posterior sampling. *Mathematics of Operations Research*, 39(4):1221–1243, 2014.
- [36] A. Guez, D. Silver, and P. Dayan. Efficient Bayes-adaptive reinforcement learning using sample-based search. In *Advances in Neural Information Processing Systems*, 2012.
- [37] M.L. Littman, A.R. Cassandra, and L.P. Kaelbling. Learning policies for partially observable environments: Scaling up. In *International Conference on Machine Learning*, 1995.
- [38] A.H. Qureshi, A. Simeonov, M.J. Bency, and M.C. Yip. Motion planning networks. In *IEEE International Conference on Robotics and Automation*, 2019.
- [39] J.J. Kuffner and S.M. LaValle. RRT-Connect: An efficient approach to single-query path planning. In *IEEE International Conference on Robotics and Automation*, 2000.